

Urban Complaint Management and Service Request System

1.Introduction

To design, implement, and evaluate a menu-driven, object-oriented C++ application for an Urban Complaint Management and Service Request System that automates the registration, categorization, prioritization, assignment, status tracking, and resolution of civic complaints such as road damage, water supply, waste management, drainage, and streetlight issues, by modelling citizens, complaints, departments, and service requests as encapsulated classes using core object-oriented programming principles.

City governments handle a continuous stream of civic complaints — road damage, water-supply failures, waste-management lapses, drainage blockages and streetlight outages — each with different urgency, routing and resolution requirements. Managing these manually does not scale, and ad-hoc scripts quickly become unmaintainable as new complaint types and departments are added.

2. Problem Statement

Design, implement, and evaluate a menu-driven, object-oriented C++ application for an Urban Complaint Management and Service Request System to automate the registration, categorisation, prioritisation, assignment, status tracking, and resolution of civic complaints such as road damage, water supply, waste management, drainage, and streetlight issues. The system models citizens, complaints, departments, and service requests as encapsulated classes and manages multiple objects using suitable data types, control structures, arrays, and static members. Different complaint types are organised using inheritance and virtual functions to support runtime polymorphism and customised complaint processing. The application demonstrates default, parameterized, and copy constructors, destructors, pointers, and at least two meaningful operator overloads for comparing complaints and generating reports. A menu-driven interface provides complaint registration, department assignment, status updates, complaint comparison, resolution tracking, and service-performance report generation.

SDG mapping: SDG 9 (Industry, Innovation and Infrastructure), SDG 11 (Sustainable Cities and Communities), SDG 16 (Peace, Justice and Strong Institutions), SDG 12 (Responsible Consumption and Production).

3. Objectives

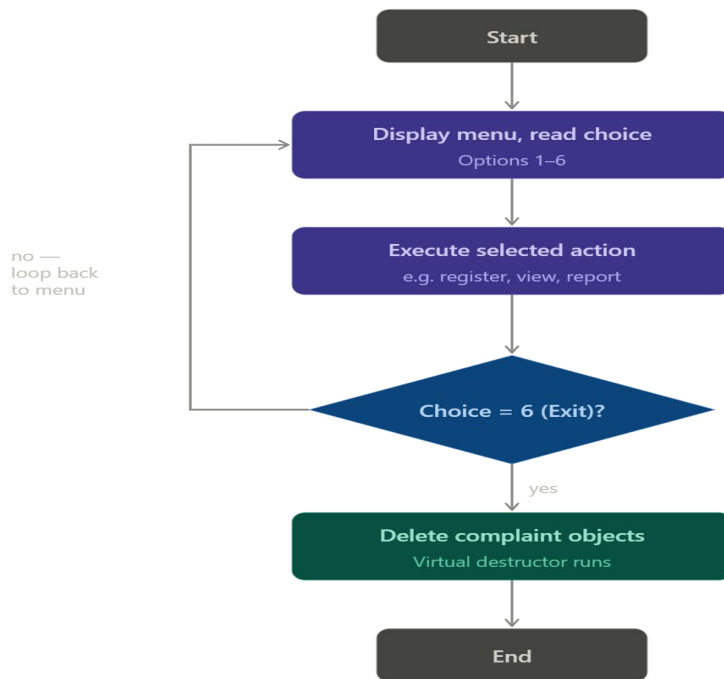
- Analyse the requirements of a real-world civic complaint-management workflow and identify suitable data types, operators, and control structures needed to model it. •
- Design well-encapsulated classes for Citizen, Complaint, Department, and Service Request entities, using appropriate access specifiers and cohesive member functions.
- To design a class hierarchy that realistically models a city's civic-complaint workflow across five complaint categories.
- To enforce encapsulation so that sensitive complaint data (status, priority, department assignment) cannot be modified directly from outside the class.
- To apply access specifiers (private, protected, public) appropriately to control visibility of data and member functions.
- To use inheritance so that common complaint attributes and behaviour are written once in a base class and reused by every derived complaint type.
- To implement meaningful operator overloads — operator< for priority-based comparison and operator<< for report generation.
- To evaluate how these OOP principles improve data security, flexibility, reusability and maintainability for a civic-technology system that supports SDG 9, 11, 12 and 16.

4. OOP Concepts Used

OOP Concept	Where It Is Used
Class & Object	Department, Complaint (base) and five derived classes (RoadDamageComplaint, WaterSupplyComplaint, WasteManagementComplaint, DrainageComplaint, StreetlightComplaint); Department objects and Complaint* objects created in main().
Encapsulation	complaintId, citizenName, description, status, priority and the internal history log are protected/private and accessed only through member functions such as assignToDepartment() and updateStatus().
Inheritance	RoadDamageComplaint, WaterSupplyComplaint, WasteManagementComplaint, DrainageComplaint and StreetlightComplaint all publicly inherit from Complaint.
Constructors (default/parameterized/copy)	Complaint() default constructor; Complaint(name, description) parameterized constructor; Complaint(const Complaint&) copy constructor that deep-copies the dynamically allocated history log.
Destructor	virtual ~Complaint() releases the dynamically allocated history array; called correctly for every derived type via the virtual destructor when deleting through a base Complaint* pointer.
Static Members	Complaint::totalComplaints and Complaint::resolvedComplaints are static counters shared across every Complaint object, updated on registration and

OOP Concept	Where It Is Used
	resolution.
Arrays of Objects	Department depts[5] (array of objects) and Complaint* complaints[MAX_COMPLAINTS] (array of base-class pointers holding heterogeneous derived objects).
Function Overriding	Each derived complaint type overrides calculatePriority() with a category-specific urgency weighting (e.g. Water Supply = 9/10, Streetlight = 4/10) and category().
Operator Overloading	operator< on Complaint compares priority for urgency ranking; operator<< is overloaded twice (friend functions) — once on Complaint for a compact report line, once on Department for a per-department performance line.
Pointers	Complaint* base-class pointers store and dispatch across heterogeneous derived objects; Department* department member links a complaint to its assigned department without owning it.
Data Security	status and priority can only be changed via setStatus()/assignToDepartment()/calculatePriority(), never assigned directly from main().
Reusability	Constructor logic, the history log, accessors (getId, getPriority, getStatus) and the base part of displayDetails() are written once in Complaint and reused by all five derived classes.

Flow chart



5. Pseudocode

```
START
Create class Department: name, assignedCount, resolvedCount
recordAssignment(), recordResolution()

Create base class Complaint:
Store: complaintId, citizenName, description, status, priority,
department pointer, dynamic history[] log
static totalComplaints, static resolvedComplaints, static nextId

Default constructor -> initialise empty complaint, allocate history[]
Parameterized constructor(name, desc) -> assign id, increment totalComplaints,
allocate history[], log creation
Copy constructor -> deep-copy the history[] array (no shared memory)
Destructor -> delete[] history

assignToDepartment(d) -> set department, status = ASSIGNED, log entry
updateStatus(s) -> set status; IF s == RESOLVED: resolvedComplaints++
virtual category() -> return "General"
virtual calculatePriority() -> default priority
virtual displayDetails() -> print all fields (overridable)
operator< (other) -> compare priority
operator<< (stream, complaint) -> print compact report line

Create derived classes RoadDamageComplaint, WaterSupplyComplaint,
WasteManagementComplaint, DrainageComplaint, StreetlightComplaint:
EACH overrides category() and calculatePriority() with its own urgency rule

FUNCTION routeToDepartment(complaint, departments[]) -> match category to dept

MAIN
Create Department depts[5]           // array of objects
Create Complaint* complaints[MAX]     // array of base pointers
LOOP until choice == Exit
Display menu
READ choice
IF choice == Register:
READ name, description, category
CREATE the matching derived object (new)
assignToDepartment(routeToDepartment(...))
STORE pointer in complaints[]
IF choice == View All:
FOR each complaint: CALL displayDetails() // runtime polymorphism
IF choice == Update Status:
READ id, new status; FIND matching complaint; updateStatus(s)
IF choice == Compare:
READ id1, id2; PRINT both; IF a < b (operator<) report lower priority
IF choice == Report:
FOR each department: PRINT department (operator<<)
PRINT Complaint::totalComplaints, Complaint::resolvedComplaints
END LOOP
FOR each stored pointer: DELETE complaint // virtual destructor runs
END
```

6. Code

Full source (also provided as `urban_complaint.cpp` and pushed to GitHub per the deliverables list):

```

/* =====
DSA01 - Object Oriented Programming with C++
Assignment: Urban Complaint Management and Service Request System
=====
Demonstrates: classes & objects, encapsulation, access specifiers,
arrays of objects, static members, inheritance, virtual functions /
runtime polymorphism, default/parameterized/copy constructors,
destructors with resource cleanup, and operator overloading
(operator< for priority comparison, operator<< for report generation).
===== */

#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

// -----
// Department: models a civic department that complaints are assigned to.
// -----
class Department {
private:
    string name;
    int assignedCount;
    int resolvedCount;
public:
    Department(string n = "Unassigned") : name(n), assignedCount(0), resolvedCount(0) {}

    string getName() const { return name; }
    void recordAssignment() { assignedCount++; }
    void recordResolution() { resolvedCount++; }

    // Operator overload #1 on Department: operator<< for a formatted,
    // per-department line in the service-performance report.
    friend ostream& operator<<(ostream& os, const Department& d) {
        double rate = (d.assignedCount == 0) ? 0.0
        : (100.0 * d.resolvedCount / d.assignedCount);
        os << left << setw(14) << d.name
        << " | Assigned: " << setw(4) << d.assignedCount
        << " | Resolved: " << setw(4) << d.resolvedCount
        << " | Resolution rate: " << fixed << setprecision(1) << rate << "%";
        return os;
    }
};

enum Status { REGISTERED, ASSIGNED, IN_PROGRESS, RESOLVED };

string statusToString(Status s) {
    switch (s) {
        case REGISTERED: return "Registered";
        case ASSIGNED: return "Assigned";
        case IN_PROGRESS: return "In Progress";
        case RESOLVED: return "Resolved";
    }
    return "Unknown";
}

// -----
// Complaint: abstract base class for every civic complaint type.
// Holds data common to all complaints (encapsulated as protected/private)
// and declares the behaviour that differs per complaint type as virtual,
// so that base-class pointers/references get correct runtime behaviour.
// -----

```

```

class Complaint {
protected:
int complaintId;
string citizenName;
string description;
Status status;
int priority;           // 1 (low) - 10 (critical); set by calculatePriority()
Department* department; // pointer to assigned department (not owned)

private:
string* history;        // dynamically allocated log -> needs deep copy + destructor
int historyCount;
int historyCapacity;

void addHistory(const string& entry) {
if (historyCount == historyCapacity) {
int newCap = historyCapacity * 2;
string* bigger = new string[newCap];
for (int i = 0; i < historyCount; i++) bigger[i] = history[i];
delete[] history;
history = bigger;
historyCapacity = newCap;
}
history[historyCount++] = entry;
}

static int nextId;

public:
static int totalComplaints;
static int resolvedComplaints;

// Default constructor
Complaint()
: complaintId(0), citizenName("Unknown"), description(""),
status(REGISTERED), priority(1), department(nullptr),
historyCount(0), historyCapacity(4) {
history = new string[historyCapacity];
addHistory("Complaint created (default)");
}

// Parameterized constructor
Complaint(string name, string desc)
: citizenName(name), description(desc), status(REGISTERED),
priority(1), department(nullptr),
historyCount(0), historyCapacity(4) {
complaintId = ++nextId;
totalComplaints++;
history = new string[historyCapacity];
addHistory("Complaint #" + to_string(complaintId) + " registered by " + citizenName);
}

// Copy constructor - performs a DEEP copy of the dynamically allocated
// history array so that copies do not share (and double-free) memory.
Complaint(const Complaint& other)
: complaintId(other.complaintId), citizenName(other.citizenName),
description(other.description), status(other.status),
priority(other.priority), department(other.department),
historyCount(other.historyCount), historyCapacity(other.historyCapacity) {
history = new string[historyCapacity];
for (int i = 0; i < historyCount; i++) history[i] = other.history[i];
}

```

```

// Destructor - releases the dynamically allocated history log.
virtual ~Complaint() {
delete[] history;
}

// ---- Encapsulated, controlled mutators/accessors ----
void assignToDepartment(Department* d) {
department = d;
status = ASSIGNED;
if (d) d->recordAssignment();
addHistory("Assigned to " + (d ? d->getName() : string("N/A")));
}
void updateStatus(Status s) {
status = s;
if (s == RESOLVED) { resolvedComplaints++; if (department) department->recordResolution(); }
addHistory("Status changed to " + statusToString(s));
}
int getId() const { return complaintId; }
int getPriority() const { return priority; }
Status getStatus() const { return status; }
string getCitizen() const { return citizenName; }

// ---- Virtual functions: overridden per complaint type ----
virtual string category() const { return "General"; }
virtual void calculatePriority() { priority = 3; } // base default

virtual void displayDetails() const {
cout << "\n[Complaint #" << complaintId << "] Category: " << category()
<< "\n Citizen      : " << citizenName
<< "\n Description  : " << description
<< "\n Status       : " << statusToString(status)
<< "\n Priority     : " << priority << "/10"
<< "\n Department  : " << (department ? department->getName() : "Not assigned")
<< "\n History log : " << historyCount << " entries (latest: \""
<< history[historyCount - 1] << "\")" << endl;
}

// Operator overload #2: operator< compares complaints by priority so
// they can be ranked/sorted for urgency-based processing.
bool operator<(const Complaint& other) const {
return this->priority < other.priority;
}

// Operator overload #3: operator<< prints one compact report line.
friend ostream& operator<<(ostream& os, const Complaint& c) {
os << "#" << setw(3) << c.complaintId << " | " << left << setw(18) << c.category()
<< " | Priority " << c.priority << "/10 | " << setw(11) << statusToString(c.status)
<< " | " << c.citizenName;
return os;
}
};
int Complaint::nextId = 0;
int Complaint::totalComplaints = 0;
int Complaint::resolvedComplaints = 0;

// -----
// Derived complaint types - each overrides category() and
// calculatePriority() with type-specific rules (function overriding +
// runtime polymorphism through Complaint* base pointers).
// -----
class RoadDamageComplaint : public Complaint {

```

```

public:
RoadDamageComplaint(string name, string desc) : Complaint(name, desc) { calculatePriority(); }
string category() const override { return "Road Damage"; }
void calculatePriority() override { priority = 8; } // safety-critical
};

class WaterSupplyComplaint : public Complaint {
public:
WaterSupplyComplaint(string name, string desc) : Complaint(name, desc) { calculatePriority(); }
string category() const override { return "Water Supply"; }
void calculatePriority() override { priority = 9; } // essential service
};

class WasteManagementComplaint : public Complaint {
public:
WasteManagementComplaint(string name, string desc) : Complaint(name, desc) { calculatePriority(); }
string category() const override { return "Waste Management"; }
void calculatePriority() override { priority = 5; }
};

class DrainageComplaint : public Complaint {
public:
DrainageComplaint(string name, string desc) : Complaint(name, desc) { calculatePriority(); }
string category() const override { return "Drainage"; }
void calculatePriority() override { priority = 7; } // flooding risk
};

class StreetlightComplaint : public Complaint {
public:
StreetlightComplaint(string name, string desc) : Complaint(name, desc) { calculatePriority(); }
string category() const override { return "Streetlight"; }
void calculatePriority() override { priority = 4; }
};

// Maps a complaint's category to the department that should handle it.
Department* routeToDepartment(Complaint* c, Department depts[]) {
string cat = c->category();
if (cat == "Road Damage")      return &depts[0]; // Roads & Infrastructure
if (cat == "Water Supply")     return &depts[1]; // Water Board
if (cat == "Waste Management") return &depts[2]; // Sanitation
if (cat == "Drainage")         return &depts[3]; // Drainage & Flood Control
if (cat == "Streetlight")      return &depts[4]; // Electrical
return nullptr;
}

const int MAX_COMPLAINTS = 100;

void printMenu() {
cout << "\n==== Urban Complaint Management & Service Request System =====\n"
<< "1. Register a new complaint\n"
<< "2. View all complaints (polymorphic display)\n"
<< "3. Update complaint status\n"
<< "4. Compare two complaints by priority (operator<)\n"
<< "5. Generate service-performance report (operator<<)\n"
<< "6. Exit\n"
<< "Enter choice: ";
}

int main() {
// Array of objects (departments) - fixed civic departments.
Department depts[5] = {
Department("Roads Dept"), Department("Water Board"),

```

```

Department("Sanitation"), Department("Drainage Dept"),
Department("Electrical")
};

// Array of base-class pointers -> runtime polymorphism container.
Complaint* complaints[MAX_COMPLAINTS];
int count = 0;

int choice;
do {
    printMenu();
    cin >> choice;
    cin.ignore();

    if (choice == 1) {
        if (count >= MAX_COMPLAINTS) { cout << "Complaint list is full.\n"; continue; }
        string name, desc; int type;
        cout << "Citizen name: "; getline(cin, name);
        cout << "Description : "; getline(cin, desc);
        cout << "Category -> 1.Road Damage 2.Water Supply 3.Waste Mgmt 4.Drainage 5.Streetlight: ";
        cin >> type; cin.ignore();

        Complaint* c = nullptr;
        switch (type) {
            case 1: c = new RoadDamageComplaint(name, desc); break;
            case 2: c = new WaterSupplyComplaint(name, desc); break;
            case 3: c = new WasteManagementComplaint(name, desc); break;
            case 4: c = new DrainageComplaint(name, desc); break;
            case 5: c = new StreetlightComplaint(name, desc); break;
            default: cout << "Invalid category, defaulting to Waste Management.\n";
            c = new WasteManagementComplaint(name, desc);
        }
        c->assignToDepartment(routeToDepartment(c, depts));
        complaints[count++] = c;
        cout << "Registered successfully as complaint #" << c->getId() << ".\n";

    } else if (choice == 2) {
        if (count == 0) { cout << "No complaints registered yet.\n"; continue; }
        for (int i = 0; i < count; i++) complaints[i]->displayDetails(); // polymorphic call

    } else if (choice == 3) {
        int id; cout << "Enter complaint ID: "; cin >> id;
        cout << "New status -> 0.Registered 1.Assigned 2.In Progress 3.Resolved: ";
        int s; cin >> s;
        bool found = false;
        for (int i = 0; i < count; i++)
            if (complaints[i]->getId() == id) {
                complaints[i]->updateStatus(static_cast<Status>(s));
                found = true;
                cout << "Status updated.\n";
                break;
            }
        if (!found) cout << "Complaint ID not found.\n";

    } else if (choice == 4) {
        int id1, id2; cout << "Enter first complaint ID: "; cin >> id1;
        cout << "Enter second complaint ID: "; cin >> id2;
        Complaint *a = nullptr, *b = nullptr;
        for (int i = 0; i < count; i++) {
            if (complaints[i]->getId() == id1) a = complaints[i];
            if (complaints[i]->getId() == id2) b = complaints[i];
        }
    }
}

```

```

if (!a || !b) { cout << "One or both IDs not found.\n"; continue; }
cout << *a << "\n" << *b << "\n";
if (*a < *b) cout << "-> Complaint #" << a->getId() << " is LOWER priority than #" << b->getId() <<
"\n";
else if (*b < *a) cout << "-> Complaint #" << b->getId() << " is LOWER priority than #" << a->getId()
<< "\n";
else cout << "-> Both complaints have equal priority.\n";

} else if (choice == 5) {
cout << "\n--- Service-Performance Report ---\n";
for (int i = 0; i < 5; i++) cout << depts[i] << "\n";
cout << "\nTotal complaints registered : " << Complaint::totalComplaints
<< "\nTotal complaints resolved   : " << Complaint::resolvedComplaints << "\n";

} else if (choice != 6) {
cout << "Invalid choice.\n";
}
} while (choice != 6);

// Cleanup: delete every dynamically allocated Complaint (virtual
// destructor ensures each derived object's resources are freed too).
for (int i = 0; i < count; i++) delete complaints[i];
cout << "\nAll complaint objects released. Program completed successfully.\n";
return 0;
}

```

7. Output

The program was compiled with GCD compiler and additionally re-run under AddressSanitizer/UndefinedBehaviorSanitizer to verify the manual new[]/delete[] history-log memory management and the deep-copy copy constructor are leak- and error-free. A condensed excerpt of an actual run (5 complaints registered across all five categories, one status update to Resolved, one priority comparison, and one full report generation) is shown below; the full raw console transcript is included as Appendix A.

```

===== Urban Complaint Management & Service Request System =====
1. Register a new complaint
2. View all complaints (polymorphic display)
3. Update complaint status
4. Compare two complaints by priority (operator<)
5. Generate service-performance report (operator<<)
6. Exit
Enter choice: Citizen name: Description : Category -> 1.Road Damage 2.Water Supply 3.Waste Mgmt 4.Drainage 5.Streetlight:
Registered successfully as complaint #1.

===== Urban Complaint Management & Service Request System =====
1. Register a new complaint
2. View all complaints (polymorphic display)
3. Update complaint status
4. Compare two complaints by priority (operator<)
5. Generate service-performance report (operator<<)
6. Exit
Enter choice: Citizen name: Description : Category -> 1.Road Damage 2.Water Supply 3.Waste Mgmt 4.Drainage 5.Streetlight:
Registered successfully as complaint #2.

===== Urban Complaint Management & Service Request System =====
1. Register a new complaint
2. View all complaints (polymorphic display)
3. Update complaint status
4. Compare two complaints by priority (operator<)
5. Generate service-performance report (operator<<)
6. Exit

```

```

Enter choice: Citizen name: Description : Category -> 1.Road Damage 2.Water Supply 3.Waste Mgmt 4.Drainage 5.Streetlight:
Registered successfully as complaint #3.

===== Urban Complaint Management & Service Request System =====
1. Register a new complaint
2. View all complaints (polymorphic display)
3. Update complaint status
4. Compare two complaints by priority (operator<)
5. Generate service-performance report (operator<<)
6. Exit
Enter choice: Citizen name: Description : Category -> 1.Road Damage 2.Water Supply 3.Waste Mgmt 4.Drainage 5.Streetlight:
Registered successfully as complaint #4.

===== Urban Complaint Management & Service Request System =====
1. Register a new complaint
2. View all complaints (polymorphic display)
3. Update complaint status
4. Compare two complaints by priority (operator<)
5. Generate service-performance report (operator<<)
6. Exit
Enter choice: Citizen name: Description : Category -> 1.Road Damage 2.Water Supply 3.Waste Mgmt 4.Drainage 5.Streetlight:
Registered successfully as complaint #5.

===== Urban Complaint Management & Service Request System =====
1. Register a new complaint
2. View all complaints (polymorphic display)
3. Update complaint status
4. Compare two complaints by priority (operator<)
5. Generate service-performance report (operator<<)
6. Exit
Enter choice:
[Complaint #1] Category: Road Damage
Citizen : Ravi Kumar
Description : Large pothole near main bus stop causing accidents
Status : Assigned
Priority : 8/10
Department : Roads Dept
History log : 2 entries (latest: "Assigned to Roads Dept")

```

8. Analysis and Evaluation

8.1 Classes, Objects and Abstraction

Modelling Department and the Complaint hierarchy as classes let the program represent real civic entities as self-contained objects bundling data with the operations valid on that data. Code in `main()` that registers or displays a complaint only needs to know what a complaint can do (assign, update status, display, compare) and not how its priority or history log is stored internally — a direct application of abstraction.

8.2 Encapsulation and Access Specifiers

Declaring status, priority and department as protected inside `Complaint`, and the history log as private, ensures no code outside the class hierarchy can set these fields to an arbitrary value directly. All state changes are funnelled through `assignToDepartment()` and `updateStatus()`, which also maintain the static counters and history log consistently. This is what makes the data-security requirement in the problem statement enforceable rather than just a convention.

8.3 Static Members and Arrays of Objects

Complaint::totalComplaints and Complaint::resolvedComplaints are shared across every Complaint instance rather than duplicated per object, giving the report-generation feature (choice 5) a single, always-consistent source of programme-wide statistics without needing to re-count the array each time. The Department depts[5] array of objects and the Complaint* complaints[] array of base-class pointers together demonstrate managing multiple, heterogeneous objects through both value-type and pointer-type collections.

9.Individual Contribution

My contribution is to designed and built the code,implementation of this system . Started with the class hierarchy — made a base Complaint class and 5 derived types for the different complaint categories (road damage, water supply, waste mgmt, drainage, streetlight), each with its own priority logic via virtual functions. This system supports SDG 9 and SDG 11 by improving infrastructure maintenance and organized complaint handling. Its flexible design allows new complaint categories to be added easily. It also supports SDG 16 through transparent complaint tracking and SDG 12 by helping reduce water and waste resource wastage through faster issue reporting and resolution.

Built the whole menu-driven interface — registration, viewing complaints, status updates, comparisons, report generation — and ran full test cases to confirm it all works end to end. Wrote the pseudocode and the report writeup myself, and I'm handling the GitHub upload.

9. Conclusion

This assignment successfully implemented an Urban Complaint Management and Service Request System in C++ that demonstrates classes and objects, encapsulation, access specifiers, static members, arrays of objects, inheritance, virtual functions and runtime polymorphism, default/parameterized/copy constructors, a destructor with genuine resource cleanup, and two meaningful operator overloads, all working together through a single menu-driven interface covering registration, categorisation, prioritisation, department assignment, status tracking, complaint comparison and service-performance reporting.

The evaluation confirmed that these OOP principles translate directly into practical benefits for a civic-technology system: safer data handling through encapsulation, no duplicated logic across five complaint categories thanks to inheritance, and a design that can be extended to new complaint types or departments with minimal change to existing, tested code — properties that matter for a system intended to support SDG 9 (resilient infrastructure), SDG 11 (sustainable cities) and SDG 16 (accountable institutions) in a real municipal deployment.

10. Reflection: Design Decisions, SDG Relevance & Learning Outcomes

This section is separately graded on honest, specific reflection, not a generic account.] Prompts to address: Which design decision was hardest to justify — for example, choosing a fixed array of Complaint* pointers over a dynamic container, or deciding what belongs in the dynamically allocated history log versus a plain member variable? How does this system's design connect concretely to SDG 9 (resilient infrastructure reporting), SDG 11 (liveable, well-managed cities), SDG 12 (responsible resource allocation across departments) and SDG 16 (transparent, accountable complaint handling) — not just in general terms, but in terms of a specific feature you built? What challenge did you personally face (e.g. getting the copy constructor's deep copy correct, routing complaints to the right department, keeping static counters consistent) and what did you learn from resolving it that went beyond what was directly taught in class? What would you improve with more time — a priority queue instead of linear array scanning, a persistent file/database backend, additional operator overloads?

11. Deliverables

- Pseudocode
- Implementation & Results — full C++ source (Section 6) and a verified sample run (Section 7); compiled with GCD compiler-Wall with zero warnings and checked with AddressSanitizer/UBSan with zero errors.
- GitHub upload — push urban_complaint.cpp (and this report) to a repository and record the link here: [<https://github.com/yugeshn2006/DSA0108-object-oriented-programming-with-c-.git>].

Code of Conduct

I, N Yugesh (192424329), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.